

Opracowanie

System e-commerce sklepu odzieżowego

Kacper Gumulak & Damian Judka

30.05.2026

Część 2: Identyfikacja Aktorów, Użycia i Dynamika

2.1 Identyfikacja głównych aktorów systemu

Architektura systemu wyraźnie oddziela uprawnienia i zakres obowiązków poprzez zastosowanie ról. Główne interakcje z systemem e-commerce realizowane są przez dwóch podstawowych aktorów (ludzkich) oraz dwa systemy zewnętrzne:

- **Customer (Klient):** Aktor zewnętrzny korzystający ze sklepu. Jego głównym celem jest przeglądanie asortymentu (widok `v_product_details`), zarządzanie własnym koszykiem (`cart`, `cart_item`), zarządzanie adresami (`addresses`, `billing_details`) oraz finalizacja zamówień.
- **User (Pracownik Sklepu / Administrator):** Aktor wewnętrzny. Zależnie od przypisanej roli (tabele `roles`, `permissions`), pracownik zarządza katalogiem produktów, monitoruje zdarzenia magazynowe (`Event Sourcing` - `stock_events`) oraz realizuje procesy logistyczne związane z wysyłką (`prepare_order_shipment`, `ship_order`) i zwrotami (`return_order`).
- **Payment Provider (System Płatności):** Aktor systemowy obsługujący transakcje finansowe i weryfikujący poprawność płatności.
- **Shipping Provider (Firma Kurierska):** Aktor systemowy odbierający zlecenia wysyłki i dostarczający numery listów przewozowych (`tracking URL`).

2.2 Wyodrębnienie i opisanie przypadków użycia (Use Cases)

Logika biznesowa została silnie osadzona w warstwie bazy danych poprzez procedury składowane, które bezpośrednio mapują się na kluczowe Przypadki Użycia (UC):

- **UC1: Zarządzanie koszykiem:** Klient może dodawać, modyfikować ilość lub usuwać produkty (procedury: `add_product_to_cart`, `increase_quantity`, `delete_product_from_cart`). System w tle waliduje stany magazynowe za pomocą funkcji `check_sellable_product_quantity`.
- **UC2: Złożenie zamówienia (Checkout):** Klient podaje dane dostawy i płatności (`add_billing_details`, `add_shipping_details`). Wywołanie procedury `add_order` tworzy zamówienie o statusie *placed*.
- **UC3: Realizacja płatności:** Po zatwierdzeniu przez Payment Providera, wywoływana jest procedura `pay_order`. Zmienia ona status na *paid* i automatycznie generuje fakturę (*invoices*) oraz pozycje na fakturze (*invoice_lines*).
- **UC4: Obsługa logistyczna:** Pracownik przygotowuje paczkę do wysyłki (`prepare_order_shipment`), wprowadzając URL śledzenia, a po jej wydaniu kurierowi oznacza jako wysłaną (`ship_order`).
- **UC5: Obsługa zwrotów i anulacji:** Pracownik lub system może anulować zamówienie (`cancel_order` dla statusów przed wysyłką) lub przyjąć zwrot (`return_order` po wysyłce). Obie procedury automatycznie dopisują odpowiednie logi do `stock_events`, aktualizując widok `v_inventory`.

2.3 Analiza problemu i specyfikacja wymagań

Wymagania dla zoptymalizowanego backendu e-commerce zostały zdefiniowane następująco:

- **Wymagania Funkcjonalne:** System musi gwarantować ścisłą kontrolę dostępności towaru w czasie rzeczywistym (wyeliminowanie problemu "oversellingu" dzięki Event Sourcingowi). Moduł koszyka musi automatycznie wyliczać wartość zamówienia (`calculate_cart_total_value`). Wszystkie dokumenty księgowe muszą być generowane w sposób zautomatyzowany natychmiast po zaksięgowaniu wpłaty.
- **Wymagania Niefunkcjonalne:** System musi izolować dane klientów. Konfiguracja wdrażanego środowiska (np. Docker Compose) musi opierać się na bezpiecznych zmiennych środowiskowych, eliminując ryzyko wycieku hardcodowanych poświadczeń. Przeniesienie kluczowej logiki na silnik InnoDB bazy MariaDB 11 zapewnia zgodność z regułami ACID i transakcyjność operacji.

Część 3: Architektura i dokładna struktura systemu

3.1 Architektura wdrożenia (Deployment Model)

System został zaprojektowany z myślą o pełnej konteneryzacji, co gwarantuje spójność między środowiskiem deweloperskim a produkcyjnym. Ostateczna architektura opiera się na technologii Docker i orkiestracji za pomocą Docker Compose, co pozwala na łatwe wdrażanie oraz izolację procesów. Architektura wdrożeniowa składa się z następujących węzłów (Nodes):

- **Klient (Przeglądarka Web):** Urządzenie końcowe użytkownika (Customer) lub administratora (User), które komunikuje się z systemem za pośrednictwem protokołu HTTP/HTTPS.
- **Kontener Aplikacji (Frontend/Middleware):** Środowisko uruchomieniowe hostujące interfejs graficzny zbudowany w oparciu o framework Python Streamlit. Kontener ten pełni rolę lekkiego klienta, który nie przechowuje stanu, a jedynie pośredniczy w wywoływaniu logiki biznesowej.
- **Kontener Bazy Danych (Backend):** Serwer MariaDB 11. To serce całego systemu. Z racji przeniesienia większości logiki biznesowej do bazy (Stored Procedures), ten węzeł realizuje najcięższe zadania obliczeniowe i transakcyjne (silnik InnoDB).
- **Wolumeny Danych (Docker Volumes):** Fizyczne, persystentne mapowanie danych bazy MariaDB na dysku hosta, zapewniające trwałość informacji o zamówieniach, logach magazynowych (Event Sourcing) i użytkownikach nawet w przypadku restartu kontenerów.

Komunikacja między kontenerem aplikacji a bazą danych odbywa się w odizolowanej sieci wewnętrznej (Docker Network), do której nie ma bezpośredniego dostępu z zewnątrz, co znacznie podnosi bezpieczeństwo całego wdrożenia.

3.2 Ostateczna struktura komponentów systemu

Architektura oprogramowania została zorganizowana w oparciu o komponenty (Component Model). Zamiast klasycznego modelu trójwarstwowego (Frontend, Backend API, Baza Danych), w naszym rozwiązaniu zintegrowano warstwę logiki biznesowej bezpośrednio z warstwą danych. Kluczowe komponenty to:

- **Moduł Interfejsu (Streamlit UI):** Odpowiada za prezentację widoków (np. `v_product_details`, `v_categories_tree`) oraz zbieranie danych z formularzy.
- **Komponent Integracyjny (DB Connector):** Łącznik w języku Python inicjujący sesje i autoryzujący dostęp do bazy.
- **Moduł Logiki Biznesowej (Stored Procedures):** Zbiór procedur składowanych wewnątrz MariaDB, realizujący konkretne zadania, takie jak `add_order()`, `pay_order()`, `prepare_order_shipment()`.
- **Moduł Składowania (Data Storage):** Ustrukturyzowane tabele relacyjne, obsługujące integralność referencyjną i przechowujące ostateczny stan systemu.

Diagram Przypadków Użycia

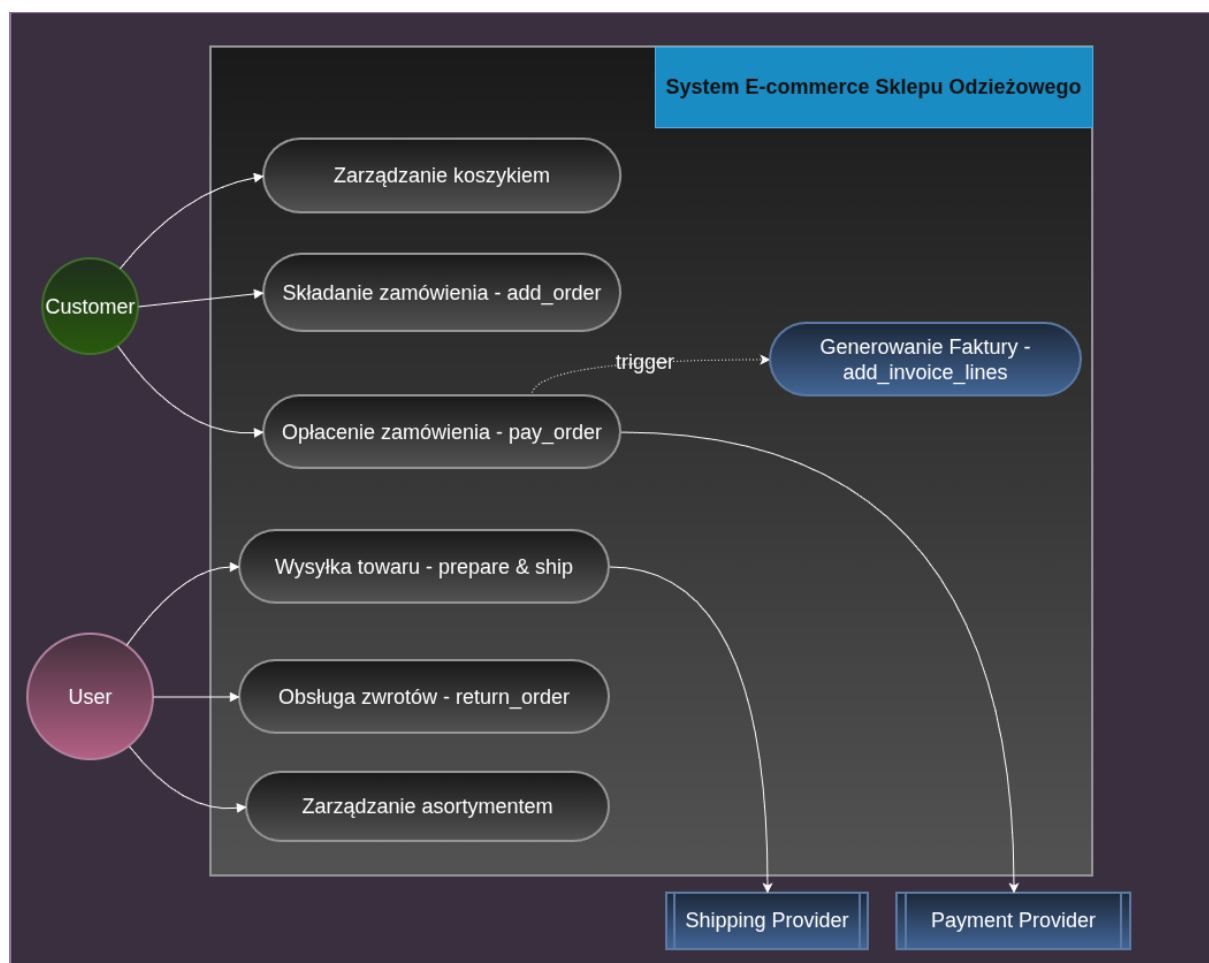


Diagram Czynności

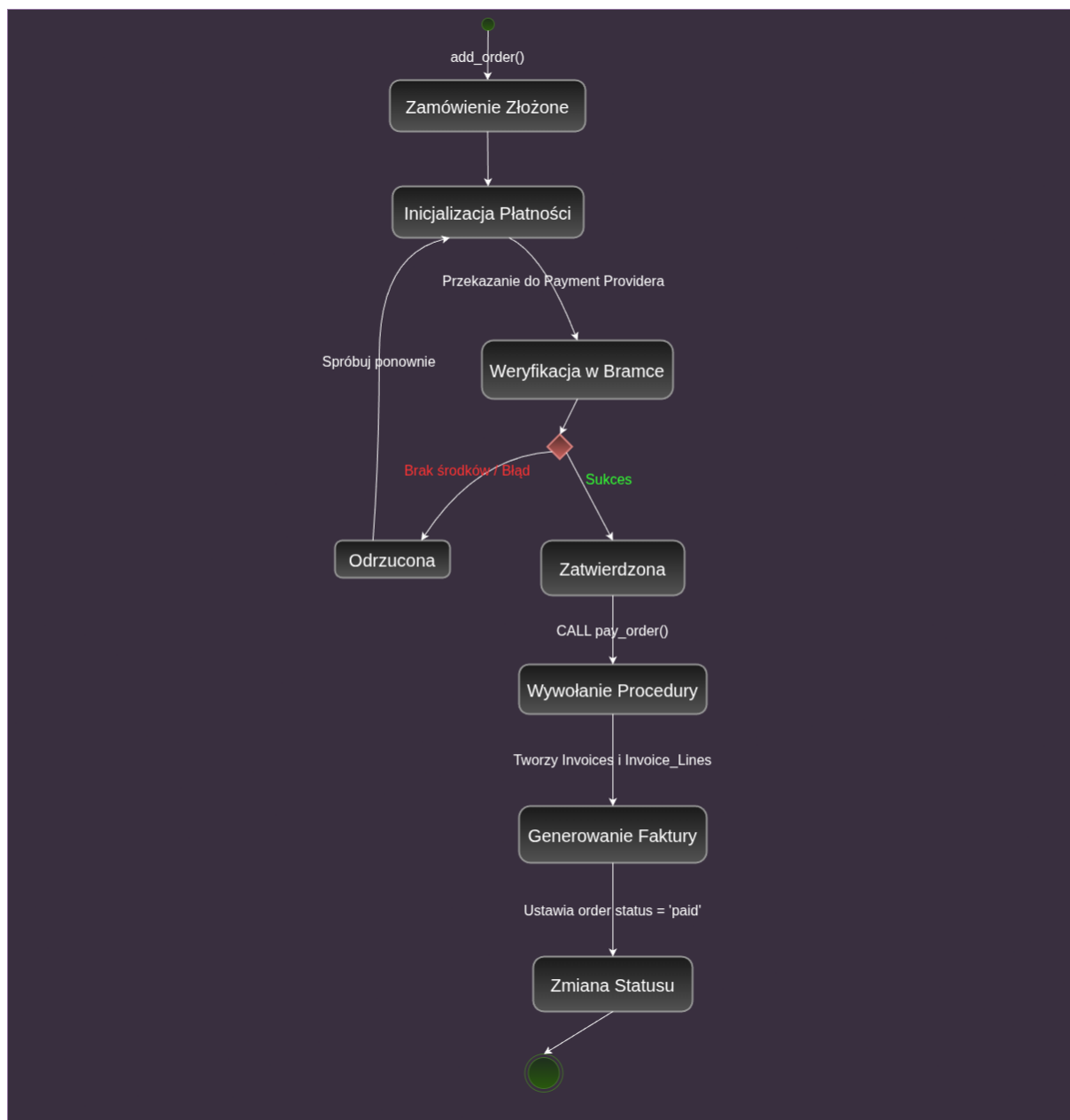


Diagram Interakcji

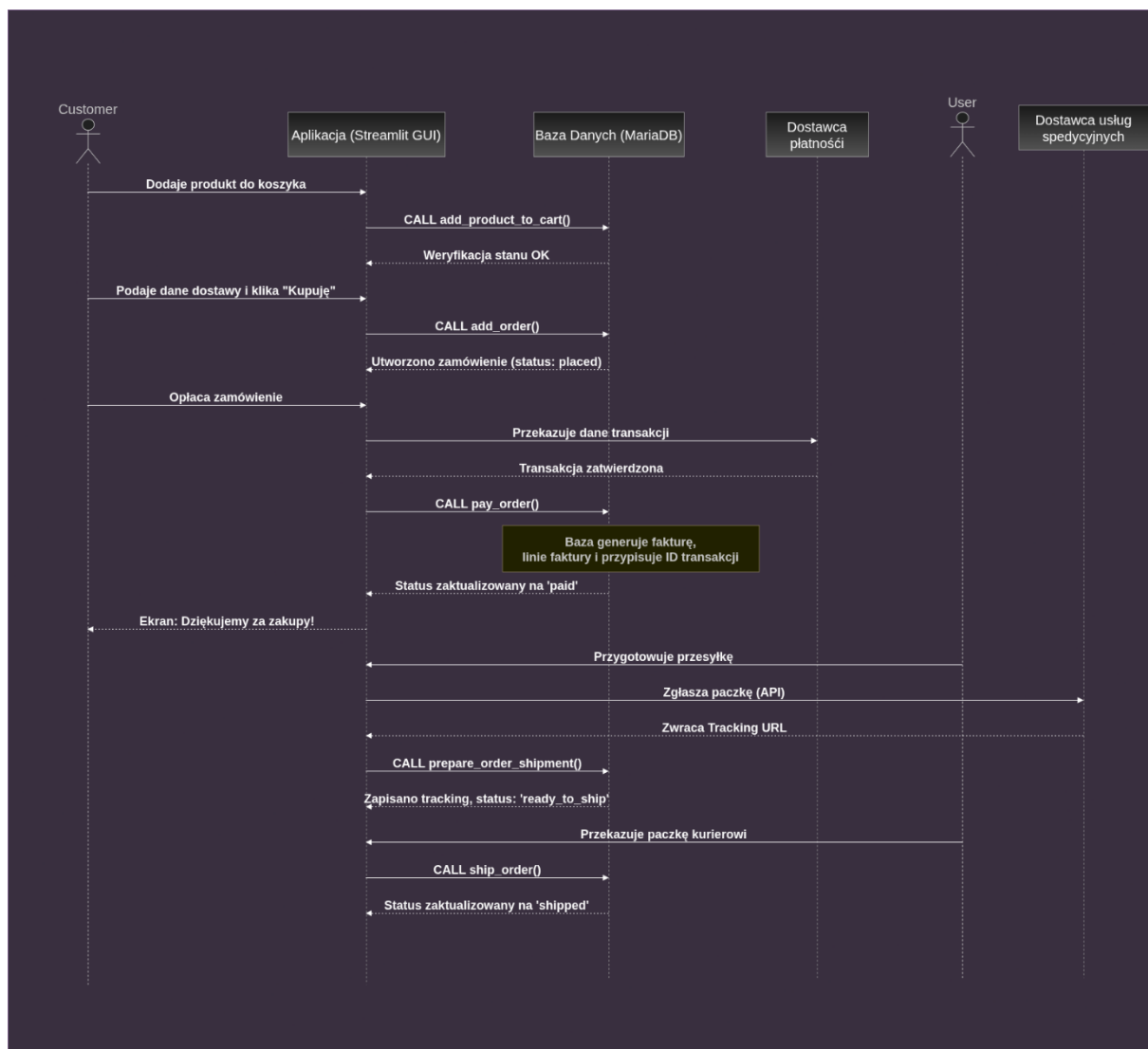


Diagram Wdrożenia

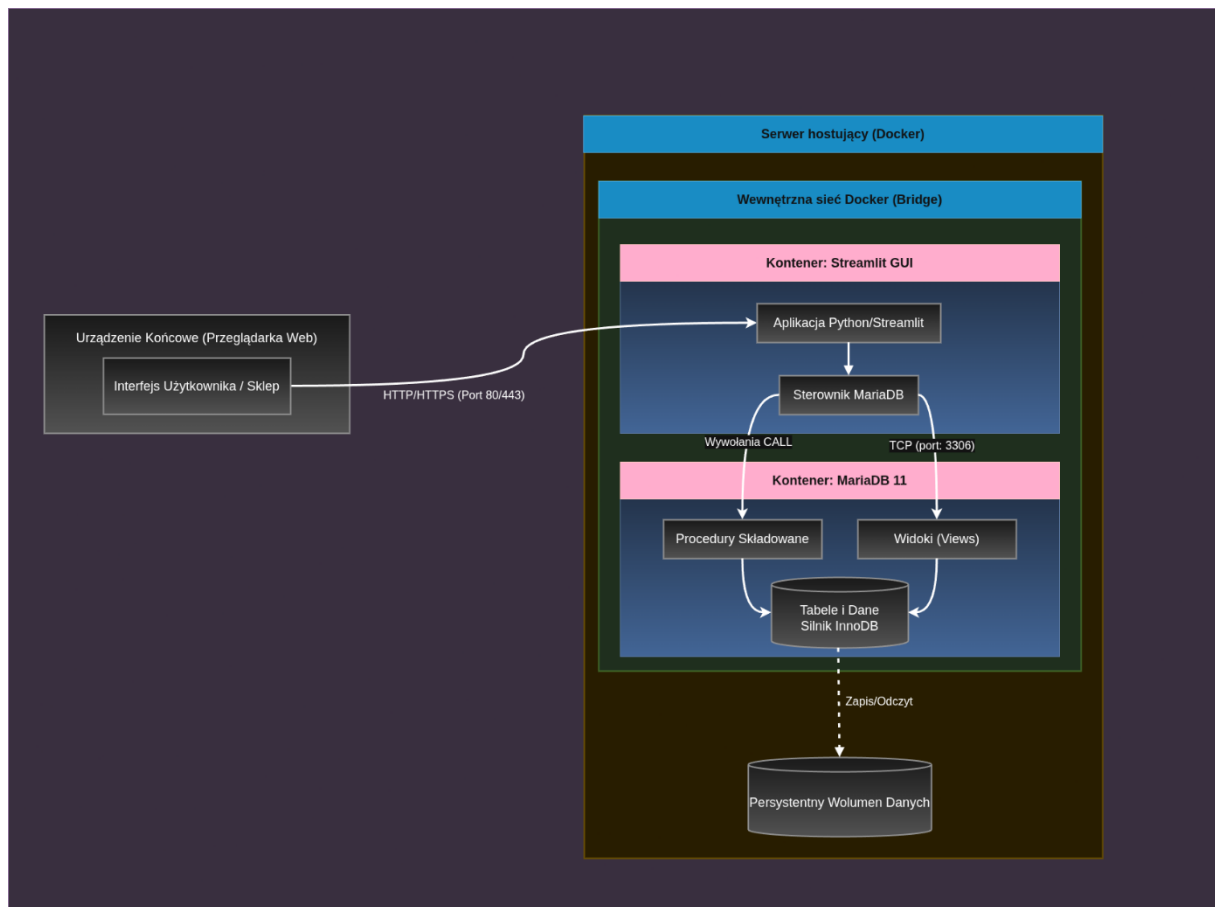


Diagram Komponentów

